

UNIVERSITY OF BUCHAREST

FACULTY OF MATHEMATICS AND COMPUTER SCIENCE

DATA SCIENCE



Practical Machine Learning
Second Project

Unsupervised flower image classification

Student

Lungu Laura-Vanesa

Bucharest, January 2026

Contents

1 Introduction **2**

2 Dataset Description and Preprocessing **2**

3 Feature Set 1 **2**

3.1 Histogram of Oriented Gradients (HOG) 3
3.2 Local Binary Patterns (LBP) 3
3.3 Color Histograms 3
3.4 Feature Combination 3
3.5 Evaluation Metrics 4
3.6 Dimensionality Reduction with PCA 4
3.7 BIRCH Clustering 5
3.7.1 Hyperparameter Tuning 5
3.8 K-Medoids Clustering 5
3.8.1 Hyperparameter Tuning 5
3.9 Test Set Results and Baseline Comparison 6

4 Feature Set 2 **7**

4.1 Dense SIFT Descriptors 7
4.2 Visual Vocabulary Construction 7
4.3 Bag of Visual Words Representation 7
4.4 TF-IDF Weighting 7
4.5 Color Histogram 8
4.6 Feature Combination 8
4.7 BIRCH Clustering 8
4.7.1 Hyperparameter Tuning 8
4.8 K-Medoids Clustering 8
4.8.1 Hyperparameter Tuning 9
4.9 Test Set Results 9
4.10 Baseline Comparison 9
4.11 Final Comparison 9

5 Cluster Interpretation **10**

6 Conclusions **11**

1 Introduction

This project explores an unsupervised flower image classification task using two clustering algorithms: BIRCH and K-Medoids. Two different feature representations are compared: a handcrafted combination of HOG, LBP, and color histograms, and a richer representation based on Dense SIFT with a Bag of Visual Words model.

The clustering results are evaluated using standard metrics and compared against both a random baseline and a supervised SVM classifier, which serves as an upper bound. The goal is to understand how feature design affects unsupervised clustering performance on real image data.

2 Dataset Description and Preprocessing

The original dataset was taken from Kaggle and contains flower images collected through web scraping. The images belong to five flower classes and vary significantly in resolution, aspect ratio, and background, which makes the task challenging.

Since the goal of this project is unsupervised classification, I decided to simplify the problem by focusing on only **three classes: rose, sunflower, and tulip**. These classes appear easier to separate visually while still preserving a realistic level of difficulty due to natural similarities in color and structure, as well as highly diverse backgrounds.

Because the dataset was created automatically through web scraping, it contained a noticeable amount of noise. I manually removed clear outliers and irrelevant images to obtain a cleaner subset. After this filtering step, the final dataset consisted of 1652 images, with a moderate class imbalance that reflects the original data distribution.

All images were center-cropped and resized to a fixed resolution of 256×256 pixels. The data was then split using a **stratified 70/15/15 train-validation-test split**, which preserves the class proportions across splits and was reused across all experiments to ensure consistent and fair evaluation.

3 Feature Set 1

The first feature set is based on classical handcrafted image features. The main idea behind this approach is to describe each image using simple and interpretable visual cues related to shape, texture, and color. These features provide a reasonable baseline for the unsupervised clustering experiments.

3.1 Histogram of Oriented Gradients (HOG)

HOG features are used to capture shape and edge information from the images by encoding the distribution of local gradient orientations. In this setup, HOG is computed on grayscale images using **9 orientation bins**, a **cell size of** 16×16 **pixels**, and 2×2 **cells per block**. Block normalization is performed using the `L2-Hys` scheme, and `transform_sqrt` is enabled to reduce the effect of illumination changes. This configuration provides a good balance between descriptive power and feature dimensionality, while keeping the computational cost reasonable.

3.2 Local Binary Patterns (LBP)

LBP features are used to capture local texture information by comparing each pixel with its neighbors and encoding local intensity patterns. A **uniform LBP** configuration is used with a **radius of 2 pixels** and **16 sampling points**. For each image, a normalized histogram with **18 bins** is computed. This single-scale setup keeps the representation compact while still capturing relevant texture details such as petal surfaces and inner flower regions.

3.3 Color Histograms

Color information is represented using **HSV color histograms**. The HSV color space is preferred over RGB because it separates chromatic information from intensity, making it more robust to lighting variations. For each image, histograms are computed independently for the H, S, and V channels, using **32 bins per channel**. The three histograms are then concatenated and L2-normalized to obtain the final color feature vector.

3.4 Feature Combination

Each feature type is extracted independently and normalized separately. HOG features are standardized using `StandardScaler`, while LBP and color histograms are L2-normalized. The final feature vector is obtained by concatenating the three feature blocks. Empirically chosen weights are applied during concatenation, with **60% weight for HOG**, **20% for LBP**, and **20% for color features**. This weighting reflects the higher importance of shape information observed during preliminary experiments.

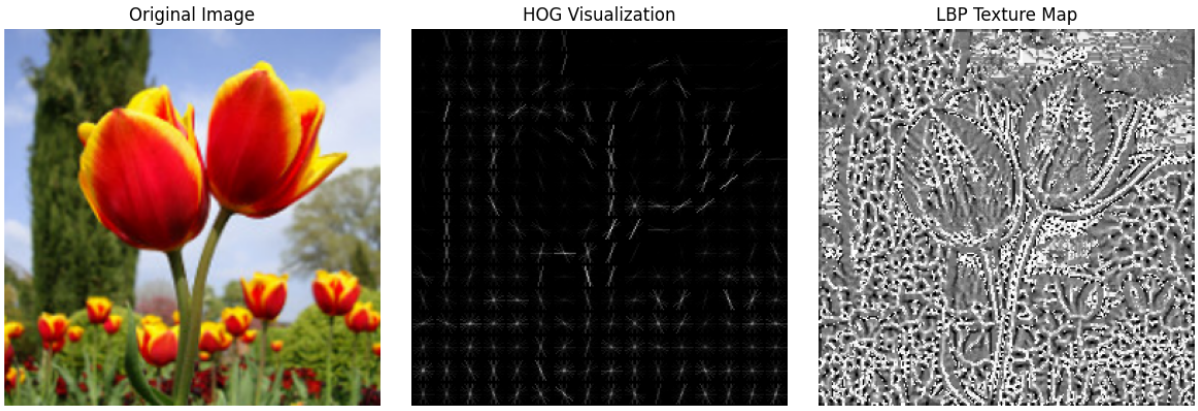


Figure 1: Example visualization of handcrafted features. From left to right: original image, HOG visualization highlighting edge and shape information, and LBP map capturing local texture patterns.

3.5 Evaluation Metrics

To evaluate the clustering results, I used 4 metrics that capture different aspects of performance.

Cluster Accuracy is the most intuitive metric. Since clusters do not have predefined label names, I first compute the best possible mapping between cluster IDs and true classes using the Hungarian algorithm, and then report the resulting accuracy.

Adjusted Rand Index (ARI) compares the predicted clustering with the ground truth by checking whether pairs of samples are grouped together consistently. The score is adjusted for chance, so ARI $= 0$ corresponds to random clustering, while ARI $= 1$ indicates perfect agreement. ARI is used as the main metric for hyperparameter selection.

Normalized Mutual Information (NMI) measures how much information is shared between the predicted clusters and the true labels. It ranges from 0 to 1, with higher values indicating better alignment.

Silhouette Score does not rely on ground-truth labels and instead measures how well-separated the clusters are in the feature space. In this project, the values are generally low (around 0.0–0.1), which is expected for real image data where different classes often overlap in feature space.

3.6 Dimensionality Reduction with PCA

Before clustering, I also experimented with PCA (Principal Component Analysis) to reduce the dimensionality of the feature vectors. The original representation is very high-dimensional, mainly due to HOG features, which can make distance-based clustering less effective. PCA projects the data onto directions of maximum variance, keeping only the most informative components. I tested several values (30, 50, 100, and 150) to find a good trade-off between preserving useful information and reducing noise.

3.7 BIRCH Clustering

BIRCH is a hierarchical clustering algorithm that incrementally builds a tree structure to summarize the data. Instead of storing all points, it groups nearby samples into compact subclusters, which makes it efficient and scalable. The main hyperparameter of BIRCH is the **threshold**, which controls the maximum radius of a subcluster. Smaller values lead to tighter and more numerous subclusters, while larger values allow more distant points to be grouped together.

3.7.1 Hyperparameter Tuning

A grid search was performed over PCA dimensions and threshold values. The model was trained on the training set and evaluated on the validation set, with configurations ranked by metric.

Table 1: Top 5 BIRCH configurations on validation set

Rank	Config	Accuracy	ARI	Silhouette
1	PCA(100), thr=0.3	58.9%	0.237	0.076
2	PCA(100), thr=0.5	58.9%	0.237	0.076
3	PCA(100), thr=0.7	58.9%	0.237	0.076
4	PCA(30), thr=1.5	59.7%	0.156	0.130
5	PCA(30), thr=0.3	59.3%	0.149	0.129

An interesting observation is that for PCA(100), changing the threshold does not affect the clustering result. This suggests that, in this reduced space, BIRCH consistently finds the same cluster structure across a wide range of threshold values.

3.8 K-Medoids Clustering

K-Medoids is similar to K-Means, but instead of using the mean of each cluster as its center, it selects a real data point (the medoid) as the cluster representative. This makes the method more robust to outliers and allows direct inspection of representative samples. Another important advantage of K-Medoids is that it supports different distance metrics. In this project, Euclidean, cosine, and Manhattan distances were tested.

3.8.1 Hyperparameter Tuning

A grid search over PCA dimensions and distance metrics was performed. The results show that both choices have a significant impact on performance.

The best configuration according to ARI uses **PCA(30)** with **Euclidean distance**, achieving **61.3% accuracy** and **ARI = 0.259**. This indicates that a small amount of dimensionality reduction helps Euclidean distance capture more meaningful structure.

At the same time, the highest accuracy (62.9%) is obtained using **cosine distance without PCA**, but with a slightly lower ARI (0.239). This suggests that cosine similarity is more robust in high-dimensional spaces, while Euclidean distance benefits more from dimensionality reduction.

Table 2: Top 5 K-Medoids configurations on validation set (sorted by ARI)

Rank	Config	Accuracy	ARI	Silhouette
1	PCA(30), euclidean	61.3%	0.259	0.059
2	PCA(50), euclidean	60.9%	0.239	0.044
3	No PCA, cosine	62.9%	0.239	0.017
4	PCA(30), cosine	59.3%	0.224	0.083
5	PCA(100), euclidean	56.5%	0.194	0.047

Overall, K-Medoids slightly outperforms BIRCH on this feature set, but both methods show moderate performance, reflecting the inherent difficulty of clustering flower images using handcrafted features.

3.9 Test Set Results and Baseline Comparison

After selecting the best configurations on the validation set, I ran the final evaluation on the test set to see how well the models generalize. BIRCH with PCA(100) and threshold 0.3 gets **58.1% accuracy** and an ARI of 0.218. K-Medoids with PCA(30) and Euclidean distance scores slightly lower at **56.0% accuracy**, but interestingly has a higher ARI (0.240). This means the clusters it finds actually align better with the true classes, even if the raw accuracy number is lower - a good reminder that accuracy alone is not enough in clustering.

To put these numbers in perspective, I compared them against two baselines. A random classifier (DummyClassifier) that just guesses achieves **31.9%**. On the other end, a supervised SVM trained on the same features reaches **75.0%**, which gives us a rough upper bound for what's achievable with these features.

So the unsupervised methods land somewhere in between: significantly better than random guessing, but still about 15–20% behind what's possible with labeled data.

Table 3: Test set results and baseline comparison for Feature Set 1 (HOG + LBP + Color)

Method	Accuracy	ARI	NMI	Type
Random (Dummy)	31.9%	–	–	Baseline
BIRCH	58.1%	0.218	0.233	Unsupervised
K-Medoids	56.0%	0.240	0.213	Unsupervised
SVM (RBF, tuned)	75.0%	–	–	Supervised

4 Feature Set 2

The second feature set is based on local keypoint descriptors and a Bag of Visual Words (BoVW) representation. Compared to Feature Set 1, this approach captures finer local details and spatial variations, but comes with higher computational cost.

4.1 Dense SIFT Descriptors

SIFT descriptors are used to capture local gradient patterns that are invariant to scale and rotation. Instead of relying on automatically detected keypoints, a dense extraction strategy is used. Dense SIFT computes descriptors on a regular grid of keypoints with fixed spacing and patch size. This ensures that all regions of the image contribute to the final representation and avoids missing important structures in low-contrast areas such as petals or smooth flower regions. In this setup, keypoints are placed on a uniform grid with a step size of **16 pixels** and a patch size of **48 pixels**. Each keypoint produces a **128-dimensional SIFT descriptor**. The same grid configuration is applied to all images to ensure consistent feature extraction.

4.2 Visual Vocabulary Construction

Since each image contains a large number of local descriptors, a visual vocabulary is built to obtain a compact and fixed-length representation. All SIFT descriptors extracted from the training set are clustered using **MiniBatch K-Means**. To keep the process computationally feasible, a random subset of descriptors is used when the total number exceeds a predefined limit. The final vocabulary size is fixed to **250 visual words**, and the same vocabulary is reused for training, validation, and test images.

4.3 Bag of Visual Words Representation

After building the vocabulary, each SIFT descriptor is assigned to its nearest visual word. Each image is then represented as a histogram counting the occurrences of visual words. This Bag of Visual Words (BoVW) representation converts a variable number of local descriptors into a fixed-length feature vector of size **250**.

4.4 TF-IDF Weighting

To reduce the influence of very frequent visual words that appear in many images, **TF-IDF weighting** is applied to the BoVW histograms. This weighting scheme increases the importance of more discriminative visual words and down-weights common patterns that carry less information.

4.5 Color Histogram

While SIFT descriptors capture texture and shape information, they do not encode color. To complement the BoVW representation, a global HSV color histogram is extracted for each image. For each HSV channel, a histogram with **32 bins** is computed and concatenated, resulting in a **96-dimensional** color feature vector.

4.6 Feature Combination

Both the TF-IDF weighted BoVW vectors and the color histograms are **L2-normalized**. The final feature vector is obtained by concatenating the two representations and is used as input for the same clustering and evaluation pipeline as in Feature Set 1, enabling a fair comparison between the two approaches.

4.7 BIRCH Clustering

BIRCH was applied to Feature Set 2 using different PCA dimensions and threshold values. The best configuration on the validation set uses **PCA(30)** with a **threshold of 0.7**, achieving **66.9% accuracy** and an **ARI of 0.280**. This represents a clear improvement compared to Feature Set 1, indicating that the Dense SIFT + BoVW representation captures more meaningful structure for hierarchical clustering.

4.7.1 Hyperparameter Tuning

A grid search was performed over PCA dimensions and threshold values. Configurations were ranked using ARI on the validation set.

Table 4: Top 5 BIRCH configurations on validation set (Feature Set 2)

Rank	Config	Accuracy	ARI	Silhouette
1	PCA(30), thr=0.7	66.9%	0.280	0.055
2	PCA(30), thr=0.3	67.3%	0.259	0.076
3	PCA(50), thr=0.3	55.6%	0.108	0.051
4	PCA(50), thr=0.5	56.0%	0.099	0.072
5	PCA(150), thr=0.7	49.2%	0.093	0.050

4.8 K-Medoids Clustering

K-Medoids was evaluated using different PCA dimensions and distance metrics. The best configuration on the validation set uses **PCA(50)** with **cosine distance**, achieving **61.3% accuracy** and an **ARI of 0.180**. Compared to BIRCH, K-Medoids benefits less from Feature Set 2.

4.8.1 Hyperparameter Tuning

Configurations were ranked by ARI on the validation set.

Table 5: Top 5 K-Medoids configurations on validation set (Feature Set 2)

Rank	Config	Accuracy	ARI	Silhouette
1	PCA(50), cosine	61.3%	0.180	0.069
2	No PCA, euclidean	52.0%	0.136	0.027
3	PCA(30), cosine	52.0%	0.134	0.041
4	PCA(100), cosine	53.2%	0.116	0.061
5	PCA(50), euclidean	53.6%	0.098	0.035

4.9 Test Set Results

The best configurations selected on the validation set were evaluated on the test set.

BIRCH (PCA(30), threshold 0.7) achieves **63.7% accuracy**, with an **ARI of 0.243** and **NMI of 0.205**. This confirms that the improved validation performance generalizes well to unseen data.

K-Medoids (PCA(50), cosine) reaches **61.3% accuracy** on the test set, but with a lower **ARI of 0.174**, indicating weaker alignment between clusters and true labels.

4.10 Baseline Comparison

Both unsupervised methods clearly outperform the random baseline (DummyClassifier), which achieves only **31.9% accuracy**.

As an upper bound, a supervised SVM with an RBF kernel was tuned on the validation set. The best configuration (**C=10**, **gamma=0.1**) achieves **85.5% accuracy** on the test set, highlighting the gap between supervised and unsupervised approaches.

4.11 Final Comparison

A summary of the test set results for Feature Set 2 is shown in Table 6. Overall, Feature Set 2 leads to a significant improvement for BIRCH compared to Feature Set 1, while K-Medoids shows more modest gains.

Table 6: Test set results and baseline comparison for Feature Set 2 (Dense SIFT + BoVW + Color)

Method	Accuracy	ARI	NMI	Type
Random (Dummy)	31.9%	–	–	Baseline
BIRCH	63.7%	0.243	0.205	Unsupervised
K-Medoids	61.3%	0.174	0.175	Unsupervised
SVM (RBF, tuned)	85.5%	–	–	Supervised

5 Cluster Interpretation

To better understand what the clustering algorithm learns, I analyzed the composition of each cluster produced by the best-performing model (Feature Set 2 + BIRCH with PCA=30, threshold=0.7).

Figure 2 shows a 2D PCA projection of the training data, comparing the true labels (left) with the predicted clusters (right). While the classes overlap significantly in the feature space, the clustering still manages to capture meaningful structure.

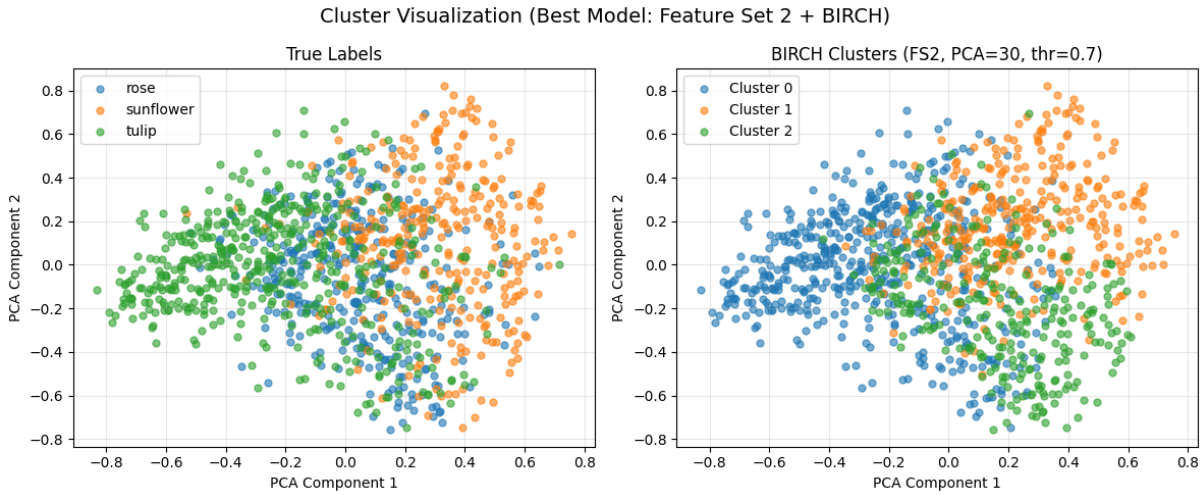


Figure 2: 2D PCA projection of Feature Set 2. Left: true class labels. Right: BIRCH cluster assignments. The overlap between classes explains the moderate clustering accuracy.

Looking at the class distribution within each cluster (Table 7), a clear pattern emerges:

- **Cluster 0** is dominated by tulip images (71%), with some roses mixed in (24%). This cluster likely captures flowers with elongated petal shapes and similar color distributions.
- **Cluster 1** primarily contains sunflowers (66%), which makes sense since sunflowers have distinctive features: large yellow petals, dark centers, and characteristic circular shapes that set them apart from roses and tulips.
- **Cluster 2** mostly groups rose images (62%), though it also includes some sunflowers and tulips. Roses often have more complex petal arrangements with reddish tones and may even come in multiple colors, which the features partially capture.

Table 7: Class distribution within each BIRCH cluster (Feature Set 2, training set)

Cluster	Rose	Sunflower	Tulip
Cluster 0	24.2%	4.8%	**71.0%**
Cluster 1	18.2%	**66.1%**	15.8%
Cluster 2	**61.7%**	21.6%	16.7%

The main source of confusion is between roses and tulips, which often share similar colors (red, pink, white) and smooth petal textures. Sunflowers, on the other hand, are more distinctive due to their unique color (yellow) and structural features (dark center, radial petals), which explains why Cluster 1 achieves the highest purity.

This analysis shows that even without access to labels during training, the unsupervised clustering discovers meaningful groupings that largely correspond to the true flower categories. The imperfect separation reflects the genuine visual similarity between certain flower types rather than a failure of the method.

6 Conclusions

This project investigated unsupervised image clustering using different feature representations and clustering algorithms. The experiments show that more expressive features lead to better clustering performance, with Dense SIFT and BoVW clearly outperforming simpler handcrafted features.